ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri

Ders Projesi

Ad-Soyad: Derda Enes Tuncer

Öğrenci No: 21291136

GitHub: github.com/derdatuncer/sql-projects

Proje 1 Video: www.youtube.com/watch?v=j72ECIzw1_s

Proje 2 Video: www.youtube.com/watch?v=1rDNclw-mCU

Contents

1. Proje 1 – Veritabanı Optimizasyonu 3

1.1 Projenin Amacı ve Kapsamı 3

1.2 Kullanılan Teknolojiler ve Araçlar 3

1.3 Test Ortamının Hazırlanması 4

1.4 İndeks Yönetimi ve Performans Kıyaslaması 5

1.4.1 Baseline (İndeks Öncesi) Performans Analizi 5

1.4.2 DMV ile Eksik İndeks (Missing Index) Tespiti 6

1.4.3 Önerilen İndeksin Uygulanması ve İyileşme 6

1.4.4 DMV ile İndeks Kullanımının İzlenmesi (Usage Stats) 7

1.5 Sorgu İyileştirme 7

1.6 Disk Parçalanması (Fragmentation) Yönetimi 8

1.7 Veritabanı Güvenliği ve Rol Yönetimi 9

1.8 Sonuçlar ve Öneriler 10

2. Proje 2 – Veritabanı Yedekleme 11

2.1 Projenin Amacı ve Kapsamı 11

2.2 Test Ortamının Hazırlanması 11

2.3 Yedekleme Stratejileri ve Uygulanması 11

2.4 Felaket Simülasyonu ve Kuyruk Yedeği (Tail-Log) 12

2.5 Kurtarma Yöntemlerinin Karşılaştırmalı Analizi 13

2.6 Yedeklerin Doğrulanması (Verification) 14

2.7 Veritabanı Yedekleme Otomasyonu 14

2.7.1 Otomasyon Stratejisi ve Zamanlama (Scheduling) 14

2.7.2 Bakım Planı (Maintenance Plan) Yapılandırması 15

2.8 Sonuçlar ve Öneriler 16

1. Proje 1 – Veritabanı Optimizasyonu

1.1 Projenin Amacı ve Kapsamı

Modern ilişkisel veritabanı yönetim sistemlerinde, veri hacminin eksponansiyel olarak büyümesi, sorgu performansının ve sistem kaynaklarının verimli kullanımının kritik bir öneme sahip olmasına yol açmaktadır. Bu proje, WideWorldImporters örnek veritabanı altyapısı kullanılarak, büyük ölçekli veri setlerinde karşılaşılan performans darboğazlarının tespit edilmesi ve bu sorunların sistematik yaklaşımlarla çözümlenmesi amacıyla geliştirilmiştir. Çalışma kapsamında; indeks mimarisinin derinlemesine incelenmesi, sorgu yürütme planlarının (execution plan) analizi, disk parçalanmasının yönetimi ve rol bazlı erişim kontrolü (RBAC) ile güvenlik politikalarının oluşturulması gibi temel veritabanı yönetimi disiplinleri ele alınmaktadır. Elde edilen bulgular, veritabanı mimarisinde uygulanacak yapısal iyileştirmelerin, sistemin genel yanıt süresini ve kaynak tüketimini ne derece optimize edebileceğini nicel metriklerle ortaya koymayı hedeflemektedir.

1.2 Kullanılan Teknolojiler ve Araçlar

- Veritabanı Yönetim Sistemi: Microsoft SQL Server
- Geliştirme ve Analiz Ortamı: SQL Server Management Studio (SSMS)

1.3 Test Ortamının Hazırlanması

Performans testlerinin güvenilir ve tekrarlanabilir bir temele oturması adına, mevcut Sales.OrderLines tablosu kaynak alınarak, çapraz birleştirme (cross join) yöntemiyle orijinal verinin yaklaşık 50 katı büyüklüğünde, hacimli bir OrderLines_PerfDemo tablosu sentetik olarak oluşturulmuştur. Bu hacim artışı, indeks kullanımının ve sorgu maliyetlerinin gerçek dünya senaryolarına uygun bir şekilde gözlemlenebilmesini sağlamaktadır. Ayrıca, ölçümlerin bellekte önceden önbelleğe alınmış verilerden (cache) etkilenmemesi ve tamamen disk üzerinden gerçekleştirilen okumaların (physical reads) net bir biçimde analiz edilebilmesi için, her test döngüsü öncesinde DBCC DROPCLEANBUFFERS ve DBCC FREEPROCCACHE komutları çalıştırılarak bellek temizliği gerçekleştirilmiştir. Aşağıda tablo veri sayısı ve örnekleri verilmiştir.

	row_count	min_order_id	max_order_id
1	11570600	1	73595

	DemoID	OrderID	StockItemID	Quantity	UnitPrice	PickingCompletedWhen
1	1	45	164	50	112.00	2013-01-02 11:00:00.0000000
2	2	1	67	10	230.00	2013-01-01 11:00:00.0000000
3	3	2	50	9	13.00	2013-01-01 11:00:00.0000000
4	4	46	89	72	18.00	2013-01-02 11:00:00.0000000
5	5	46	171	90	32.00	2013-01-02 11:00:00.0000000
6	6	2	10	9	32.00	2013-01-01 11:00:00.0000000
7	7	47	167	50	99.00	2013-01-02 11:00:00.0000000
8	8	47	219	80	50.00	2013-01-02 11:00:00.0000000

Performans ölçümlerinin fiziksel disk dağınıklığından bağımsız olarak analiz edilebilmesi amacıyla, temel test tablosundan tamamen ayrı olarak OrderLines_FragDemo isimli ikinci bir tablo daha oluşturulmuştur. Bu tablonun ayrı tutulmasındaki temel mühendislik yaklaşımı; kasıtlı olarak oluşturulacak veri parçalanması (fragmentation) simülasyonlarının, sistemin temel I/O ve işlemci metriklerini manipüle etmesinin önüne geçmektir. Bu yalıtılmış mimari sayesinde, indeks bakım (rebuild ve reorganize) stratejileri diğer sorgu optimizasyon testlerini etkilemeden, tamamen kendi özel ortamında güvenilir bir şekilde ölçülebilmiştir.

	distinct_orders	distinct_stockitems
1	73595	227

	table_name	index_name	index_type_desc	avg_fragmentation_in_percent	avg_page_space_used_in_percent	page_count
1	OrderLines_FragDemo	CIX_OrderLines_FragDemo_OrderID_StockItemID	CLUSTERED INDEX	0	99,1406844576229	3944
2	OrderLines_FragDemo	CIX_OrderLines_FragDemo_OrderID_StockItemID	CLUSTERED INDEX	0	92,016036570299	9
3	OrderLines_FragDemo	CIX_OrderLines_FragDemo_OrderID_StockItemID	CLUSTERED INDEX	0	1,86557944156165	1

1.4 İndeks Yönetimi ve Performans Kıyaslaması

Veritabanı performansını doğrudan etkileyen en önemli unsurlardan biri veri erişim yollarıdır. Bu bölümde, verinin fiziksel olarak nasıl tarandığı ve indeks kullanımının sorgu maliyetlerine etkisi incelenmiştir.

1.4.1 Baseline (İndeks Öncesi) Performans Analizi

Herhangi bir ikincil indeks (non-clustered index) bulunmayan bir tabloda, belirli bir aralıktaki verilerin getirilmesi istendiğinde, SQL Server tüm tabloyu baştan sona taramak zorunda kalır. Bu işlem, gereksiz sayfa okumalarına (logical reads) yol açarak I/O maliyetini ve işlemci yükünü ciddi oranda artırır. Yapılan baseline testlerinde, indeks yokluğunun sistem üzerindeki ağır yükü açıkça gözlemlenmiştir. Aşağıda hem test sorguları hem de bunların indexsiz sonuçları yer alacaktır.

Test Sorgusu 1:

```
PRINT('Q1: BASELINE RANGE QUERY (INDEXSIZ)');
SELECT OrderID, StockItemID, Quantity, UnitPrice FROM dbo.OrderLines_PerfDemo
WHERE OrderID BETWEEN 50000 AND 60000 ORDER BY OrderID, StockItemID;
GO
```

SQL Server Çalıştırma İstatistikleri:

CPU time = 3796 ms, elapsed time = 8143 ms, Logical Read: 192851

Test Sorgusu 2:

```
PRINT('Q2: BASELINE AGGREGATE QUERY (INDEXSIZ)');
SELECT StockItemID, COUNT(*) AS line_count, SUM(Quantity) AS total_quantity, AVG(UnitPrice) AS avg_unit_price
FROM dbo.OrderLines_PerfDemo WHERE OrderID BETWEEN 50000 AND 70000 GROUP BY StockItemID ORDER BY StockItemID;
GO
```

SQL Server Execution Times:

CPU time = 2764 ms, elapsed time = 380 ms, Logical Read: 192851

Yukarıda verilen queryler cache’de yer edinebilsin diye defalarca çalıştırılmıştır.

1.4.2 DMV ile Eksik İndeks (Missing Index) Tespiti

Maliyetli sorgu sonrası sistemin tepkisi incelenmiştir. SQL Server, sys.dm_db_missing_index_details ve sys.dm_db_missing_index_groups DMV'leri üzerinden sorgulanarak, eksik indeks önerileri raporlanmıştır. Sistem sadece eksik indeksi bildirmekle kalmamış, aynı zamanda performansa sağlayacağı tahmini yüzde etkisini (Avg User Impact) ve gerekli CREATE INDEX scriptini (örneğin OrderID ve StockItemID kapsayan yapı) dinamik olarak üretmiştir.

Aşağıda sırasıyla önce en maliyetli sorguların incelendiği DMV daha sonra da eksik index önerileri DMV'si sonuçları verilmiştir.

	execution_count	total_cpu_ms	total_elapsed_ms	total_logical_reads	avg_cpu_ms	avg_elapsed_ms	avg_logical_reads	total_physical_reads	total_logical_writes	query_text
1	16	64117	122347	3232036	4007	7646	202002	279579	764	SELECT [OrderID],[StockItemID],[Quantity],[UnitPrice] F...
2	11	33117	4166	2121746	3010	378	192886	0	0	SELECT StockItemID, COUNT(*) AS line_count, SUM(...

	database_name	table_name	user_seeks	user_scans	avg_total_user_cost	avg_user_impact	suggested_index
1	WideWorldImporters	OrderLines_PerfDemo	27	0	160,498536173656	83,49	CREATE INDEX IX_OrderLines_PerfDemo_ ON [WideWor...

Önerilen İndeks: CREATE INDEX IX_OrderLines_PerfDemo_ ON [WideWorldImporters].[dbo].[OrderLines_PerfDemo] ([OrderID]) INCLUDE ([StockItemID], [Quantity], [UnitPrice])

SQL Server'ın önerdiği bu index, WideWorldImporters içindeki OrderLines_PerfDemo tablosunda OrderID kolonu üzerinden arama yapan sorguları hızlandırmak için oluşturuluyor; OrderID index'in ana kolonu (key) olurken, StockItemID, Quantity ve UnitPrice kolonları INCLUDE edilerek sorguların ihtiyaç duyduğu veriler doğrudan index'ten karşılanıyor, böylece tabloya geri dönmeye gerek kalmadan daha az IO ile daha hızlı sonuç elde ediliyor.

1.4.3 Önerilen İndeksin Uygulanması ve İyileşme

Sistemin önerdiği kapsayan indeks (Covering Index) uygulanmış ve referans sorgu tekrar çalıştırılmıştır. Sonuç olarak, tam tablo taraması (Scan) yerini hedefli indeks aramasına (Index Seek) bırakmış, veri okuma maliyetleri keskin bir biçimde düşmüştür. Aşağıda indeks uygulanması sonucunda daha önce verilen test sorgularının çalıştırma istatistikleri verilmiştir.

Sorgu 1 SQL Server Çalıştırma İstatistikleri:

CPU time = 438 ms, elapsed time = 7334 ms, logical reads 6790

Sorgu 2 SQL Server Çalıştırma İstatistikleri:

CPU time = 1436 ms, elapsed time = 214 ms, logical reads 13706

Bu çalışmada WideWorldImporters veritabanından türetilen test tablosu üzerinde yapılan ölçümlerde, önerilen index'in uygulanmasıyla birlikte özellikle logical read ve CPU sürelerinde ciddi düşüşler gözlemlenmiştir; bu da sorguların daha az veri okuyarak daha verimli çalıştığını göstermektedir. Ancak test verisinin performans farkını belirginleştirmek amacıyla aynı kayıtların çok sayıda tekrar edilmesiyle oluşturulmuş olması, veri dağılımını yapay hale getirmiş ve elde edilen kazanımların gerçek sistemlere kıyasla daha abartılı görünmesine neden olmuştur. Buna rağmen sonuçlar, doğru tasarlanmış bir index'in sorgu performansını iyileştirmede önemli bir rol oynadığını açıkça ortaya koymaktadır.

1.4.4 DMV ile İndeks Kullanımının İzlenmesi (Usage Stats)

Oluşturulan indeksin gerçekten kullanılıp kullanılmadığını doğrulamak için sys.dm_db_index_usage_stats görünümü incelenmiştir. Sorgu motorunun oluşturulan indeksi tercih ettiği, user_seeks sayacının artışıyla matematiksel olarak kanıtlanmıştır.

	table_name	index_name	type_desc	user_seeks	user_scans	user_lookups	user_updates	last_user_seek	last_user_scan
1	OrderLines_PerfDemo	IX_OrderLines_PerfDemo_OrderID_StockItemID_Cover...	NONCLUSTERED	12	0	0	0	2026-04-21 05:55:13.923	NULL
2	OrderLines_PerfDemo	NULL	HEAP	0	31	0	0	NULL	2026-04-21 05:30:18.427

1.5 Sorgu İyileştirme

Sorgu yazım şeklinin indeks kullanımını nasıl etkilediği (SARGability) incelenmiştir. Kolon üzerinde fonksiyon kullanımı (örn: YEAR()) indeksi geçersiz kılarken, doğrudan aralık bazlı filtreleme performansı artırmıştır.

Sorgu Tipi	Yazım Biçimi	Sonuç
Non-SARGable (Kötü)	WHERE YEAR(Date) = 2016	Full Scan (Yavaş)
SARGable (İyi)	WHERE Date >= '2016-01-01' AND ...	Index Seek (Hızlı)

Sorgu İyileştirme İçin Kullanılan İndeks: CREATE NONCLUSTERED INDEX IX_OrderLines_PerfDemo_PickingCompletedWhen ON dbo.OrderLines_PerfDemo (PickingCompletedWhen) INCLUDE (DemoID, OrderID, StockItemID, Quantity, UnitPrice);

Kötü Sorgu:

```
PRINT 'Q3_AFTER_DATE_INDEX_BAD_QUERY (NON-SARGABLE)';
SELECT DemoID, OrderID, StockItemID, Quantity, UnitPrice, PickingCompletedWhen
FROM dbo.OrderLines_PerfDemo
WHERE YEAR(PickingCompletedWhen) = 2016
ORDER BY OrderID;
GO
```

Kötü Sorgu SQL Server Çalıştırma İstatistikleri:

CPU time = 3171 ms, elapsed time = 7983 ms, logical reads 68436

İyi Sorgu:

```
PRINT 'Q4_AFTER_DATE_INDEX_GOOD_QUERY (SARGABLE)';
SELECT DemoID, OrderID, StockItemID, Quantity, UnitPrice, PickingCompletedWhen
FROM dbo.OrderLines_PerfDemo
WHERE PickingCompletedWhen >= '2016-01-01' AND PickingCompletedWhen < '2017-01-01'
ORDER BY OrderID;
GO
```

İyi Sorgu SQL Server Çalıştırma İstatistikleri:

CPU time = 1922 ms, elapsed time = 7604 ms, logical reads 8714

Bu testte, YEAR(PickingCompletedWhen) = 2016 şeklinde yazılan sorgunun, tarih kolonu üzerinde fonksiyon kullanıldığı için index kullanımını engellediği ve bu nedenle yüksek logical read (68.436) ve CPU maliyetine yol açtığı görülmektedir. Buna karşılık, aynı filtre koşulunun PickingCompletedWhen >= '2016-01-01' AND < '2017-01-01' biçiminde yeniden yazılmasıyla sorgu SARGable hale getirilmiş, böylece index'ten etkin şekilde yararlanılarak logical read değeri 8.714'e kadar düşürülmüş ve CPU süresi belirgin biçimde azalmıştır. Elapsed time'da sınırlı bir iyileşme görülse de, özellikle IO maliyetindeki bu büyük düşüş, sorgu yazım biçiminin performans üzerinde doğrudan etkili olduğunu açıkça göstermektedir.

1.6 Disk Parçalanması (Fragmentation) Yönetimi

Yoğun INSERT/UPDATE işlemleri sonrasında veri sayfalarının disk üzerindeki fiziksel parçalanma oranı sys.dm_db_index_physical_stats DMV'si ile

ölçülmüştür. Parçalanmanın yüksek olduğu durumlarda performansı korumak için REORGANIZE ve REBUILD işlemleri uygulanarak disk yapısı onarılmıştır.

Aşağıda bilerek parçalanan indeks'in fiziksel durumunu gösteren DMV ile REBUILD ve REORGANIZE sonrası indeks'in fiziksel durumunu gösteren DMV sırasıyla verilmiştir.

	table_name	index_name	index_type_desc	avg_fragmentation_in_percent	avg_page_space_used_in_percent	page_count
1	OrderLines_FragDemo	CIX_OrderLines_FragDemo_OrderID_StockItemID	CLUSTERED INDEX	97,7608035994559	61,6782802075612	9557
2	OrderLines_FragDemo	CIX_OrderLines_FragDemo_OrderID_StockItemID	CLUSTERED INDEX	70	66,9273535952557	30
3	OrderLines_FragDemo	CIX_OrderLines_FragDemo_OrderID_StockItemID	CLUSTERED INDEX	0	6,27625401531999	1

	table_name	index_name	index_type_desc	avg_fragmentation_in_percent	avg_page_space_used_in_percent	page_count
1	OrderLines_FragDemo	CIX_OrderLines_FragDemo_OrderID_StockItemID	CLUSTERED INDEX	0,0168236877523553	99,1838151717321	5944
2	OrderLines_FragDemo	CIX_OrderLines_FragDemo_OrderID_StockItemID	CLUSTERED INDEX	0	96,274549048678	13
3	OrderLines_FragDemo	CIX_OrderLines_FragDemo_OrderID_StockItemID	CLUSTERED INDEX	0	2,70570793180133	1

Parçalanma simülasyonunda indeks bozulma oranının %97.76 gibi kritik bir seviyeye ulaştığı gözlemlenmiştir. Bu sorunun çözümünde, düşük seviyeli bozulmalarda tercih edilen REORGANIZE işlemi ile indeksin baştan yaratıldığı REBUILD stratejileri değerlendirilmiştir. Hasarın yüksek boyutu nedeniyle sisteme REBUILD uygulanmış; sonuç olarak parçalanma %0.016'ya düşürülerek disk okuma (I/O) performansı ve depolama verimliliği başarıyla optimize edilmiştir.

1.7 Veritabanı Güvenliği ve Rol Yönetimi

Veritabanı güvenliğini sağlamak amacıyla, "En Az Yetki Prensibi" (Least Privilege) temel alınarak Rol Bazlı Erişim Kontrolü (RBAC) mimarisi kurulmuştur. Bu kapsamda, sadece okuma yapabilen PerfDemoReader ve tüm veri manipülasyonu işlemlerine yetkili PerfDemoAdmin rolleri oluşturularak yetki sınırları sys.database_permissions üzerinden doğrulanmıştır.

	state_desc	permission_name	grantee_name	object_name
3	GRANT	INSERT	PerfDemoAdmin	OrderLines_FragDemo
4	GRANT	SELECT	PerfDemoAdmin	OrderLines_FragDemo
5	GRANT	UPDATE	PerfDemoAdmin	OrderLines_FragDemo
6	GRANT	ALTER	PerfDemoAdmin	OrderLines_PerfDemo
7	GRANT	DELETE	PerfDemoAdmin	OrderLines_PerfDemo
8	GRANT	INSERT	PerfDemoAdmin	OrderLines_PerfDemo
9	GRANT	SELECT	PerfDemoAdmin	OrderLines_PerfDemo
10	GRANT	UPDATE	PerfDemoAdmin	OrderLines_PerfDemo
11	GRANT	SELECT	PerfDemoReader	OrderLines_FragDemo
12	GRANT	SELECT	PerfDemoReader	OrderLines_PerfDemo

Kurulan mimarinin geçerliliği EXECUTE AS USER komutuyla test edilmiş; sadece okuma yetkili kullanıcının veri güncelleme girişimi sistem tarafından güvenlik ihlali gerekçesiyle reddedilirken, yönetici kullanıcının işlemi başarıyla tamamlanmıştır.

```
(12 satır etkilendi)
reader_user -> SELECT test

(5 satır etkilendi)
reader_user -> UPDATE test (basarisiz olmali)

(0 satır etkilendi)
The UPDATE permission was denied on the object 'OrderLines_PerfDemo', database 'WideWorldImporters', schema 'dbo'.
admin_user -> UPDATE test (Basarili Olmali)

(1 satır etkilendi)
UPDATE SUCCEEDED
```

Bu simülasyon, sistemdeki kritik verilerin yetkisiz müdahalelere karşı başarıyla korunduğunu kanıtlamaktadır.

1.8 Sonuçlar ve Öneriler

Bu proje kapsamında, büyük veri hacimlerine sahip sistemlerde performansın rastgele denemelerle değil, veritabanı motorunun ürettiği metriklerle (veriye dayalı olarak) nasıl optimize edileceği uygulamalı olarak kanıtlanmıştır. Çalışmadan elde edilen temel sonuçlar ve kurumsal sistemler için öneriler aşağıda özetlenmiştir:

1. **Veriye Dayalı İndeksleme (DMV Kullanımı):** Sorgu performansını artırmak için sezgisel indeksler yaratmak yerine sys.dm_db_missing_index_details DMV'si kullanılmıştır. Sistemin kendi önerdiği kapsayan indeksin (Covering Index) uygulanmasıyla, tam tablo taramaları (Index Scan) hedefli aramalara (Index Seek) dönüştürülmüş ve I/O (disk okuma) maliyetleri dramatik şekilde düşürülmüştür.

2. **Sorgu Yazım Standartları (SARGability):** İndekslerin varlığının tek başına yeterli olmadığı kanıtlanmıştır. Tarih kolonları üzerinde YEAR() fonksiyonu kullanıldığında sistemin indeksi yok saydığı; ancak tarih aralığı (>= ve <) kullanıldığında indeksi başarıyla tetiklediği gösterilerek, "SARGable" kod yazımının önemi vurgulanmıştır.

3. **Disk Parçalanması ve Fiziksel Optimizasyon:** İzole edilen OrderLines_FragDemo tablosunda %97.76 seviyesine ulaşan kritik disk parçalanması, REBUILD işlemi ile onarılarak %0.016'ya indirilmiştir. Bu onarım sadece hızı artırmakla kalmamış, verinin tutulduğu sayfa sayısını azaltarak depolama alanından da tasarruf sağlamıştır.

4. **Veritabanı Güvenliği (RBAC):** "En Az Yetki Prensibi" doğrultusunda kurulan mimaride, PerfDemoReader rolündeki kullanıcının veri güncelleme işlemi veritabanı motoru tarafından başarıyla engellenirken, sadece yetkili PerfDemoAdmin kullanıcısına izin verilmiş ve veri bütünlüğü güvence altına alınmıştır.

2. Proje 2 – Veritabanı Yedekleme

2.1 Projenin Amacı ve Kapsamı

Veritabanı sistemlerinde veri kaybı; donanım arızaları, yazılımsal sorunlar veya insan kaynaklı hatalar (örneğin kaza ile çalıştırılan bir DELETE komutu) nedeniyle yaşanabilecek kritik bir risktir. Bu projenin amacı, felaket senaryolarına karşı dayanıklı bir veritabanı altyapısı tasarlamak ve silinen verilerin "Zamana Dönüş" (Point-in-Time Restore) stratejisi ile minimum kayıpla nasıl kurtarılabileceğini uygulamalı olarak göstermektir.

2.2 Test Ortamının Hazırlanması

Kurtarma testlerinin mevcut sistemleri etkilemeden güvenle yapılabilmesi için izole bir DisasterRecoveryTest_DB veritabanı oluşturulmuştur. Point-in-Time Restore (Zamana Dönüş) işleminin yapılabilmesi için veritabanının Kurtarma Modeli zorunlu olarak FULL olarak ayarlanmıştır. Test verisi olarak MusteriHesaplari tablosu yaratılmış ve içerisine 1000 adet başlangıç eklenmiştir.

	HesapID	MusteriAd	Bakiye	OlusturmaTarihi
1	1	Müşteri_1	5567.20	2026-04-21 09:51:16.957
2	2	Müşteri_2	9735.48	2026-04-21 09:51:16.957
3	3	Müşteri_3	7521.41	2026-04-21 09:51:16.957
4	4	Müşteri_4	7451.23	2026-04-21 09:51:16.957
5	5	Müşteri_5	4033.77	2026-04-21 09:51:16.957
6	6	Müşteri_6	509.64	2026-04-21 09:51:16.957
7	7	Müşteri_7	9568.98	2026-04-21 09:51:16.957
8	8	Müşteri_8	9944.50	2026-04-21 09:51:16.957
9	9	Müşteri_9	2240.57	2026-04-21 09:51:16.957
10	10	Müşteri_10	4741.68	2026-04-21 09:51:16.957
11	11	Müşteri_11	1913.24	2026-04-21 09:51:16.957
12	12	Müşteri_12	1692.80	2026-04-21 09:51:16.957
13	13	Müşteri_13	8582.86	2026-04-21 09:51:16.957
14	14	Müşteri_14	1884.23	2026-04-21 09:51:16.957
15	15	Müşteri_15	1852.87	2026-04-21 09:51:16.957
16	16	Müşteri_16	3342.40	2026-04-21 09:51:16.960

2.3 Yedekleme Stratejileri ve Uygulanması

Oluşturulan veri seti üzerinde, kurumsal yedekleme standartlarına uygun olarak üç aşamalı bir yedekleme zinciri oluşturulmuştur:

- **Tam Yedek (Full Backup):** Sistemdeki tüm verilerin ve yapıların tam bir kopyası alınarak kurtarma zincirinin temeli atılmıştır.

- **Fark Yedeği (Differential Backup):** Tabloya 2 yeni müşteri kaydı eklendikten sonra, sadece Tam Yedekten bu yana değişen verileri içeren Fark Yedeği alınarak disk alanından ve zamandan tasarruf sağlayan model test edilmiştir.
- **İşlem Günlüğü Yedeği (Transaction Log Backup):** Tabloya 1 yeni kayıt daha eklenmiş ve sadece son işlemleri saniye saniye kaydeden Log Yedeği alınmıştır. Toplam kayıt sayısı bu aşamada 1003'e ulaşmıştır.

Aşağıda sırayla alınan yedeklemelerin başarılı iletileri verilmiştir.

```
11 percent processed.
20 percent processed.
31 percent processed.
40 percent processed.
50 percent processed.
61 percent processed.
70 percent processed.
80 percent processed.
91 percent processed.
100 percent processed.
Processed 520 pages for database 'DisasterRecoveryTest_DB', file 'DisasterRecoveryTest_DB' on file 1.
Processed 2 pages for database 'DisasterRecoveryTest_DB', file 'DisasterRecoveryTest_DB_log' on file 1.
BACKUP DATABASE successfully processed 522 pages in 0.025 seconds (162.968 MB/sec).

(1 satır etkilendi)
16 percent processed.
24 percent processed.
32 percent processed.
40 percent processed.
56 percent processed.
64 percent processed.
73 percent processed.
81 percent processed.
97 percent processed.
100 percent processed.
Processed 112 pages for database 'DisasterRecoveryTest_DB', file 'DisasterRecoveryTest_DB' on file 1.
Processed 2 pages for database 'DisasterRecoveryTest_DB', file 'DisasterRecoveryTest_DB_log' on file 1.
BACKUP DATABASE WITH DIFFERENTIAL successfully processed 114 pages in 0.015 seconds (59.114 MB/sec).

(1 satır etkilendi)
76 percent processed.
100 percent processed.
Processed 11 pages for database 'DisasterRecoveryTest_DB', file 'DisasterRecoveryTest_DB_log' on file 1.
BACKUP LOG successfully processed 11 pages in 0.002 seconds (41.015 MB/sec).

Tamamlanma süresi:2026-04-21T09:55:05.6636218+03:00
```

2.4 Felaket Simülasyonu ve Kuyruk Yedeği (Tail-Log)

Sistemin güvenilirliğini test etmek amacıyla kasıtlı bir felaket senaryosu gerçekleştirilmiştir. Sistem saati not edildikten hemen sonra, DELETE FROM dbo.MusteriHesaplari; komutu çalıştırılarak tablodaki tüm veriler (1003 kayıt) kaza ile silinmiş ve veri kaybı simüle edilmiştir.

Veritabanı kilitlenmeden ve daha fazla hareket işlenmeden hemen önce, o ana kadar olan son işlemleri (yapılan hata dahil) güvene almak için NORECOVERY modunda tail log yedeği alınmış ve veritabanı "Restoring" durumuna geçirilmiştir.

	Felaket_Gerceklesmeden_Hemen_Onceki_Zaman
1	2026-04-21 10:36:16.093

	Kalan_Kayit_Sayisi
1	0

DisasterRecoveryTest_DB (Geri yükleniy

2.5 Kurtarma Yöntemlerinin Karşılaştırmalı Analizi

Mevcut yedek zinciri kullanılarak 3 farklı kurtarma senaryosu test edilmiş ve alınan yedeklerin veri kurtarmadaki önemi ispatlanmıştır:

- **Yöntem 1: Sadece Tam (Full) Yedekten Kurtarma**

Sadece DisasterRecoveryTest_DB_FULL.bak dosyası kullanılarak yapılan kurtarma işleminde veritabanı başarıyla ayağa kaldırılmış, ancak yalnızca ilk yedeğin alındığı andaki 1000 kayıt geri dönebilmiştir. Diff ve Log yedekleri devreye sokulmadığı için aradaki 3 kayıtlık veri kaybı yaşanmıştır.

	Kalan_Kayit
1	1000

- **Yöntem 2: Fark (Differential) Yedeği ile Kurtarma**

Full yedek NORECOVERY moduyla yüklenmiş, ardından üzerine DisasterRecoveryTest_DB_DIFF.bak dosyası işlenmiştir. Bu yöntemle 1002 kayıt geri getirilmiş, yedekleme süresinin kısaltıldığı ancak son Log yedeği kullanılmadığı için en son eklenen 1 kaydın kaybedildiği gözlemlenmiştir.

	Kalan_Kayit
1	1002

- **Yöntem 3: Zamana Dönüş (Point-in-Time Restore) ve Tam Kurtarma**

Sıfır veri kaybını hedefleyen bu nihai yöntemde zincir tam olarak işletilmiştir. Sırasıyla Full, Diff ve Log yedekleri NORECOVERY ile yüklenmiş; en son adımda alınan Kuyruk Yedeği (Tail-Log) STOPAT parametresi kullanılarak sisteme işlenmiştir. Veritabanı, felaketin gerçekleştiği anın tam salisesi salisesine öncesine hedefli olarak döndürülmüş ve 1003 kaydın tamamı kayıpsız bir şekilde kurtarılmıştır.

	Kalan_Kayit
1	1003

2.6 Yedeklerin Doğrulanması (Verification)

Bir felaket anında alınan yedeklerin disk üzerinde bozuk (corrupt) çıkması büyük bir risktir. Bu projede, alınan dosyaların fiziksel bütünlüğünü test etmek için RESTORE VERIFYONLY komutu kullanılmış, diskteki .bak ve .trn dosyaları bellekte başarıyla doğrulanmıştır.

```
The backup set on file 1 is valid.
The backup set on file 1 is valid.
The backup set on file 1 is valid.
The backup set on file 1 is valid.

Tamamlanma süresi:2026-04-21T11:04:40.9383938+03:00
```

2.7 Veritabanı Yedekleme Otomasyonu

Veritabanı yönetiminde sürekliliği sağlamak ve insan kaynaklı hataları minimize etmek amacıyla, hazırlanan yedekleme stratejilerinin otomatik bir takvime bağlanması zorunludur. Bu proje kapsamında, SQL Server Management Studio (SSMS) üzerinde yer alan SQL Server Agent ve Maintenance Plans araçları kullanılarak otomasyon kurgusu tasarlanmıştır.

2.7.1 Otomasyon Stratejisi ve Zamanlama (Scheduling)

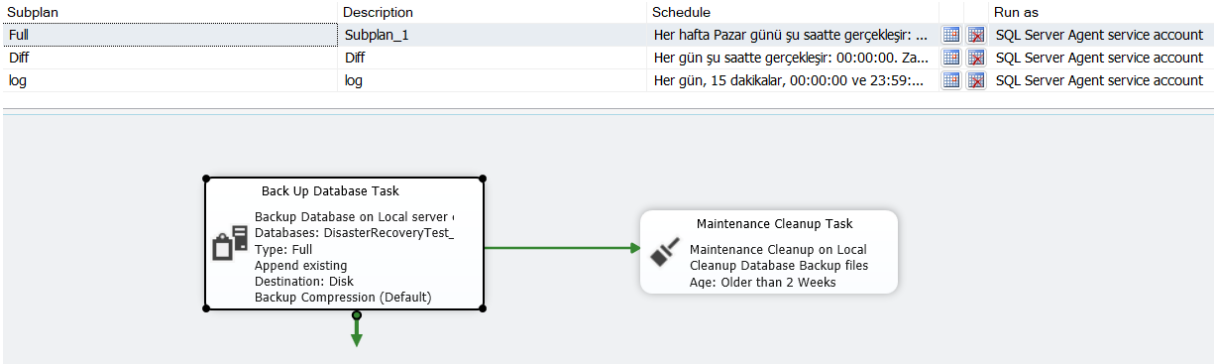
Güvenli bir geri dönüş zinciri oluşturmak için aşağıdaki hiyerarşik zamanlama planı önerilmektedir:

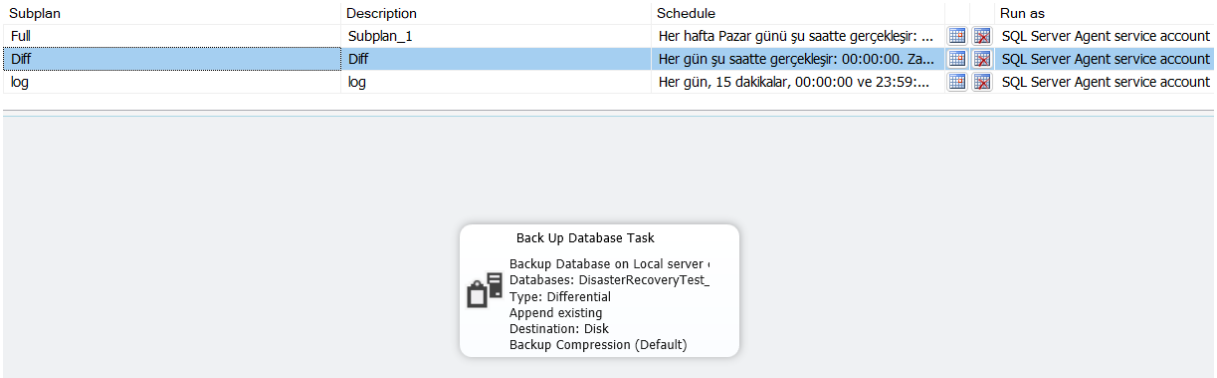
- **Full Backup (Tam Yedek):** Haftalık veya günlük (sistem yoğunluğuna göre) tam yedekleme.
- **Differential Backup (Fark Yedeği):** Günlük veya 6-12 saatlik aralıklarla, tam yedekten bu yana değişen verilerin yedeği.

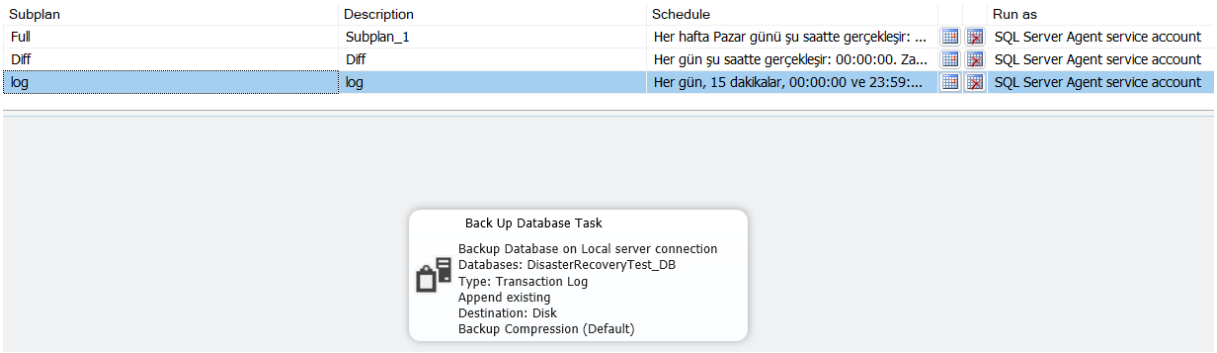
- **Transaction Log Backup (Log Yedeği):** Kritik sistemlerde 15-30 dakikalık aralıklarla saniye bazlı işlem kayıtlarının yedeği.

2.7.2 Bakım Planı (Maintenance Plan) Yapılandırması

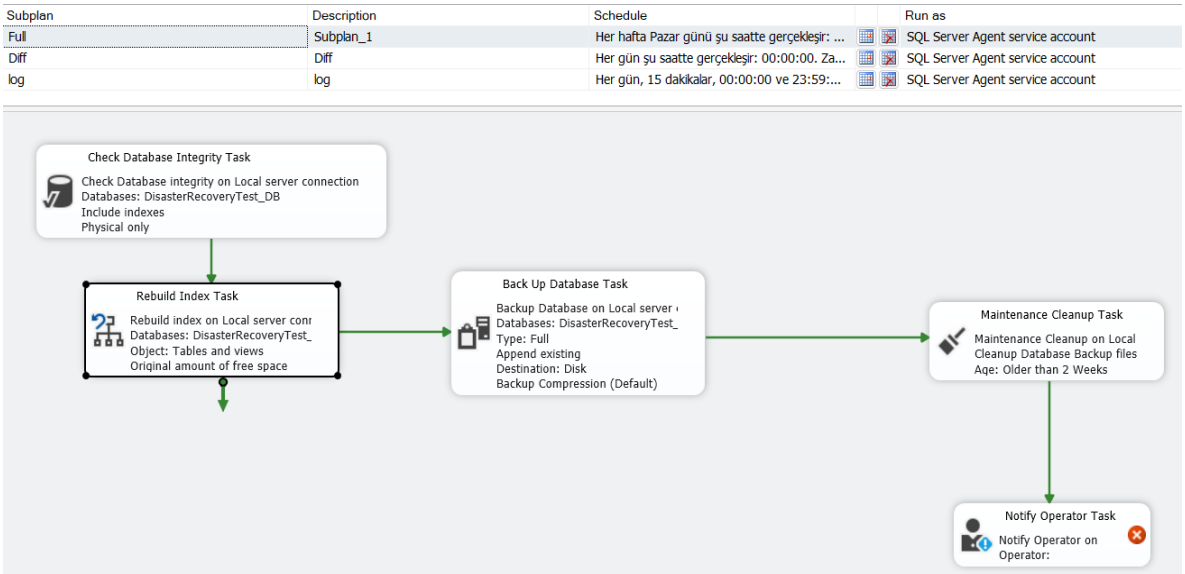
SSMS üzerinde oluşturulan Bakım Planı sayesinde; yedekleme dosyalarının otomatik olarak isimlendirilmesi, belirlenen bir klasöre aktarılması ve eski yedeklerin (örn. 30 günden eski) otomatik olarak silinmesi (Clean Up Task) sağlanmıştır. Bu sayede disk doluluk oranları kontrol altında tutulurken, felaket anında ihtiyaç duyulacak güncel yedek zinciri her an hazır bulundurulmaktadır. Aşağıda maintenance plan verilmiştir.







Üstte görülen plan yalın haldedir. Ekstra araçlarla geliştirilebilir. Aşağıda bir örneği mevcuttur.



2.8 Sonuçlar ve Öneriler

Gerçekleştirilen karşılaştırmalı simülasyonlar ve yapılandırmalar sonucunda aşağıdaki kritik çıktılar elde edilmiştir:

- **Yedekleme Zincirinin Önemi:** Sadece tam yedek almanın gün içi veri kayıplarını engelleyemediği, tam koruma için mutlaka fark ve log yedeklemelerinin sürece dahil edilmesi gerektiği kanıtlanmıştır.
- **Sıfır Veri Kaybı (Point-in-Time Restore):** Doğru yapılandırılmış FULL kurtarma modeli ve eksiksiz yedekleme zinciri (Full + Diff + Log + Tail-Log) kullanıldığında, kaza ile yapılan (örneğin hatalı DELETE işlemi) ölümcül veri silinmelerinin hedefli zamana dönüş tekniği ile kayıpsız bir şekilde geri alınabileceği görülmüştür.
- **Yedek Güvenilirliği (Verification):** Bir felaket anında alınan yedek dosyalarının bozuk (corrupt) çıkması riskine karşı RESTORE VERIFYONLY komutu projenin standart bir adımı haline getirilmiş; alınan yedeklerin fiziksel bütünlüğü bellek üzerinde doğrulanarak "kurtarma garantisi" sağlanmıştır.
- Sistem Otomasyonu ve Sürdürülebilirlik: Manuel işlemlerin taşıdığı riskleri (insan hatası, unutulma vb.) ortadan kaldırmak amacıyla süreç, Bakım Planları (Maintenance Plans) kullanılarak otomatikleştirilmiştir. Haftalık Full, Günlük Diff ve 15 dakikalık Log yedekleme periyotları kurgulanmış; ayrıca eski yedeklerin otomatik olarak silinmesi (Cleanup Task) sağlanarak

kendi kendini yöneten sürdürülebilir bir koruma mimarisi devreye alınmıştır.